Week 7: Programming larger software projects

Exercises

1)

Code answer In the lecture we discussed how we could implement an approximate sine function using the Taylor series equation:

$$\sin(x) = x - \frac{x^3}{3!} + \frac{x^5}{5!} - \frac{x^7}{7!} + \ldots$$

(a) Write this function in a C program so that it calculates the sine function with precision up to n Taylor series terms, e.g. the above example shows 4 terms. Your sine function should accept both the x value and n precision as input. The signature of your function should be:

```c
double taylor_sine(double x, int n) {
  // implement your function here
  return -1.0;
}
```

Write your Taylor series implementation of sine as a **library** consisting of: a header file and a source file (with no **main()** function).

.h file:

```c
#ifndef TAYLOR_SINE_H
#define TAYLOR_SINE_H

double taylor_sine(double x, int n);

#endif
```

.c file

```c
#include "taylor_sine.h"
#include <math.h>

// Function to calculate the factorial of a number
double factorial(int num) {
    double result = 1.0; // Initialize result to 1.0 since factorial of 0 is 1
    for (int i = 1; i <= num; i++) { // Loop from 1 to num
        result *= i; // Multiply result by i in each iteration
    }
    return result; // Return the calculated factorial value
}

// Function to calculate sine using the Taylor series
double taylor_sine(double x, int n) {
    double sine_value = 0.0; // Initialize the sine value to 0.0
    for (int i = 0; i < n; i++) { // Loop to calculate each term of the Taylor series
        int term_index = 2 * i + 1; // Calculate the exponent for the term (1, 3, 5, ...)
        // Calculate the value of the term:
        // (i % 2 == 0 ? 1 : -1) decides the sign based on whether the index is even or odd
        // pow(x, term_index) calculates x raised to the power of term_index
        // factorial(term_index) calculates the factorial of term_index
        double term = (i % 2 == 0 ? 1 : -1) * (pow(x, term_index) / factorial(term_index));
        sine_value += term; // Add the term value to sine_value
    }

  return sine_value;
}
```

(b) Write some tests for different values of x (try both small and large input values), and compare your function output with the ANSI C sin function.

```c
#include <stdio.h>
#include <math.h>         // For the ANSI C sin function
#include "taylor_sine.h"  // Include the custom sine function

int main() {
    // Array of test values for x (small and large values)
    double test_values[] = {0.0, M_PI / 6, M_PI / 4, M_PI / 2, M_PI, 2 * M_PI, 10.0, 100.0};
    int n_terms = 10; // Number of terms for the Taylor series approximation

    // Iterate through the test values
    for (int i = 0; i < sizeof(test_values) / sizeof(test_values[0]); i++) {
        double x = test_values[i];
        double taylor_result = taylor_sine(x, n_terms); // Calculate using Taylor series
        double math_result = sin(x); // Calculate using ANSI C sin function

        // Print the results for comparison
        printf("x = %.2f\n", x);
        printf("Taylor series result: %.8f\n", taylor_result);
        printf("ANSI C sin result: %.8f\n", math_result);
        printf("Difference: %.8f\n\n", fabs(taylor_result - math_result));
    }

    return 0;
}
```

(c) Answer the following questions using your test program, and please **provide your answers as comments in your test program**: Which intervals of input x did your function give a similar result to the ANSI C sin function? What impact did increasing the precision have (i.e. increasing the number of Taylor series terms)?

```c
/* 1. c) answer
1. Which intervals of input x did your function give a similar result to the ANSI C sin function?

   - The Taylor series approximation gives similar results to the ANSI C sin function when x is small,
     specifically in the range of -2π to 2π. For these values, the difference between the Taylor series
     result and the ANSI C sin function result is very small (often close to zero).
   - For larger values of x (e.g., x = 10.0 or x = 100.0), the Taylor series approximation starts to deviate
     significantly from the ANSI C sin function. This is because the Taylor series converges slowly for larger
     input values, and more terms are needed for accurate results.

2. What impact did increasing the precision have (i.e., increasing the number of Taylor series terms)?

   - Increasing the number of terms (n_terms) in the Taylor series improves the accuracy of the approximation.
     For smaller values of x, even a few terms (e.g., n_terms = 5) yield accurate results.
     However, for larger values of x, increasing the number of terms (e.g., n_terms = 20 or more) is necessary
     to reduce the difference between the Taylor series result and the ANSI C sin function.
   - The more terms included, the more accurate the result becomes, but at the cost of increased computation time.
     There's a balance between accuracy and efficiency when choosing the number of terms for different input ranges.
*/
```

2)

Code answer Stacks are containers where items are retrieved according to the order of insertion, independent of content. Stacks maintain *last-in, first-out order (LIFO)*. The abstract operations on a stack include:

Function	Description
`push(x, s)`	Insert item `x` at the top of stack `s`.
`pop(s)`	Return (and remove) the top item of stack *s*
`initialize(s)`	Create an empty stack
`full(s)`, `empty(s)`	Test whether the stack can accept more pushes or pops, respectively. *There is a trick to this, see if you can spot it!*

Note that there is no element search operation defined on standard stacks. Defining these abstract operations enables us to build a stack module to use and reuse without knowing the details of the implementation. The easiest implementation uses an array with an index variable to represent the top of the stack. An alternative implementation, using linked lists, is better because it can't overflow.

Stacks naturally model piles of objects, such as dinner plates. After a new plate is washed, it is pushed on the top of the stack. When someone is hungry, a clean plate is popped off the top. A stack is an appropriate data structure for this task since the plates don't care which one is used next. Thus

one important application of stacks is whenever **order doesn't matter**, because stacks are particularly simple containers to implement.

(a) Implement a stack based on **singly-linked lists** as discussed in the lecture.

(b) Test your implementation. **Create a new file**, where you include your stack library header file. You should expect the following "laws" to hold for any implementation of a stack. **Hint:** you should enforce these conditions using `assert` statements:

(1) After executing `initialize(s);` the stack `s` must be empty.

(2) After executing `push(x, s); y = pop(s);` the `s` must be the same as before execution of the two commands, and `x` must equal `y`.

(3) After executing `push(x0, s); push(x1, s); y0 = pop(s); y1 = pop(s);` the stack `s` must be the same as before execution of the four commands, `x0` must equal `y1`, and `x1` must equal `y0`.

Remark

Stack order is important in processing any properly nested structure. This includes parenthesised formulas (push when you see a "(" token, pop when you see a ")" token), recursive program calls (push on a procedure entry, pop on a procedure exit — we will be discussing **recursion** in Lecture 9), and depth-first traversals of graphs (push on discovering a vertex, pop on leaving it for the last time).

```c
#include <stdio.h>
#include <stdlib.h>
#include <assert.h>
#include "stack.h"

// Function to initialize the stack
void initialize(stack *s) {
  s->head = NULL;        // The stack starts empty
}

// Function to push an element onto the stack
void push(int element, stack *s) {
    node *new_node = (node*)malloc(sizeof(node)); // Allocate memory for the new node
    assert(new_node != NULL);                     // Ensure memory allocation is successful
    new_node->data = element;                     // Set the new node's data to the element value
    new_node->next = s->head;                     // Point the new node's next to the current head
    s->head = new_node;                            // Update the head of the stack to be the new node
}

int pop(stack *s) {
    assert(!empty(s));        // Ensure the stack is not empty before popping
    node *temp = s->head;     // Store the current head node in a temporary variable
    int value = temp->data;   // Get the value from the node to return
    s->head = temp->next;     // Move the head to the next node in the stack
    free(temp);               // Free the memory allocated for the old head node
    return value;             // Return the value from the popped node
}

bool empty(stack *s) {
  return s->head == NULL; // Return true if the head is NULL, indicating the stack is empty
}

// Function to check if the stack is full (always returns false for a linked list implementation)
bool full(stack *s) {
    return false; // A linked list implementation of a stack is never full unless memory is exhausted
}
```

Program testing:

```
===============================================================================
All tests passed (26 assertions in 7 test cases)
```